

摘要

电流表由电流互感器取能，次级 270 匝并设抽头，其中 170 匝经 SS16 全桥整流、齐纳箝位后向储能电容充电，再由低静态电流 LDO 稳压为 3.3 V，不接任何外部电源。取电与传感分时复用同一绕组：空闲期 PMOS、NMOS 双端关断，互感器输出全部用于储能；测量时两管导通接入采样电阻，MSPM0G3507 以内部 2.5 V 基准、4 kHz 采样率经 DMA 采集 800 点，用 Hann 加权真有效值和分档标定表换算出电流，约 260 ms 后按序掉电。创新点是以一只隔离肖特基把储能支路与采集支路解耦，使一次测量完全由电容独立供电；并以实测标定表代替名义比例系数，吸收互感器非线性。读表器为 Android 应用，经 HC-42 蓝牙以请求-应答方式读数，支持人工与 120 s 自动两种模式、越限报警和记录趋势。0.55 ~ 2.32 A 区间已建立 38 点标定表，表内拟合残差中位数 0.238%，系统在无外部供电下完成测量与无线上报。

关键词：无源电流表；MSPM0G3507；真有效值；分时取能；低功耗；BLE

一、 方案比较与论证

1. 取能与传感方案

方案一：两组独立次级绕组，分别驱动取电电路和传感电路。两条通道互相隔离，调试直观，但两组绕组同时占用磁芯窗口，绕制量大，匝数和耦合差异还会引入一致性误差。

方案二：一组带抽头的次级绕组，用受控 MOS 开关对取电和传感分时复用。空闲期断开传感前端，互感器输出全部经全桥向储能电容充电；测量窗口内 PA26 拉低使 PMOS 导通、PA8 拉高使 NMOS 导通，传感前端接入，此时系统由储能电容独立供电。该方案不重复绕组，且传感负载不在冷启动阶段占用取电功率——本题按启动电流计分，这一点直接影响得分。

本系统采用方案二。次级绕组总计 270 匝并设抽头，经不同抽头组合比较，以 170 匝有效绕组接入整流桥。

2. 整流与稳压电源方案

三个方案的前段相同：次级经全桥整流后向储能电容充电，齐纳二极管 D_Z 限制储能节点电压。箝位不可省略，线路电流达到 2.2 A 时次级开路电压远高于器件耐压。区别只在储能节点之后靠什么得到 3.3 V，电路如附录图 3 所示。

方案一：储能节点直接作为工作电压，不设稳压器。器件最少，静态电流只有齐纳的漏电流。但箝位点即工作电压，储能电容只能充到 3.3 V 附近，而电容储能与端电压平方成正比，同容值下可存能量被压到最小。

方案二：箝位点取在远高于 3.3 V 处，储能电容工作在该电压，再由低静态电流 LDO 降到 3.3 V。储能电压提高之后同容值电容存的能量按平方增加，LDO 静态电流可做到微安级，不需要电感，输出无开关纹波。代价是线性稳压搬运的是电荷而不是能量，压差部分在稳压器上变成热。

方案三：稳压改用降压型开关变换器。变换器按功率守恒工作，同一段放电能交付 $\eta \cdot \frac{1}{2} C_s (V_H^2 - V_L^2)$ ，高压段的能量也取得出来。但变换器从储能电压刚开始建立时就在耗电：输入低于其最小启动电压期间它并不工作，却仍有静态电流和开关节点漏电，把冷启动最紧张的那点取电功率消耗掉。

本系统采用方案二。冷启动阶段取电功率只有毫瓦量级，稳压器自身在这一阶段的消耗直接决定电流表能不能启动，而启动电流与启动时间合计 40 分，权重高于稳压效率；LDO 在储能电压尚未建立时几乎不取电，方案三则相反。测量窗口的能量由储能电容按式（4）承担，线性稳压的效率损失不影响单次测

量能否完成。

3. 电流有效值测量与量程方案

方案一：外部偏置与外部运放。由独立基准把交流采样信号偏置到 ADC 输入范围中点，再经外部运放放大。电路独立、调试直观，但需增加运放、偏置基准和增益切换器件，占板面积并带来额外静态功耗。

方案二：使用 MSPM0G3507 片内 OPA 和 DAC。DAC 产生可调偏置，片内 OPA 完成放大，由软件配置 1、2、4、8、16、32 倍增益并按采样幅值自动切换。外围器件少、静态功耗低，偏置和增益可由程序协同调整。

本系统采用方案二，并用 ADC 采样序列计算真有效值，不使用整流滤波后测平均值再折算的做法——后者只对正弦有效，而标定要对标的是标准表的真有效值读数。

二、 参数计算、系统设计与实现

1. 电路关键参数计算

忽略励磁电流与损耗时，互感器原、副边安匝平衡，

$$N_1 I_1 = N_2 I_2, \quad (1)$$

故副边电流为

$$I_2 = \frac{N_1}{N_2} I_1. \quad (2)$$

取 $N_1 = 5$ 、接入匝数 $N_2 = 170$ ，原边 2.0 A 对应副边 58.8 mA。增加匝数可提高低电流下的感应电压，同时也增加绕组电阻和分布参数，故做成 270 匝带抽头、按实测选定接入匝数的结构。

桥式整流每半周有两只二极管串联导通。设次级瞬时电压为 $u_2(t)$ 、单只二极管正向压降为 V_F ，则整流后瞬时电压为

$$u_r(t) = \max(|u_2(t)| - 2V_F, 0). \quad (3)$$

输入电压越低， $2V_F$ 占比越大，故全桥和隔离二极管一律选肖特基。

储能电容要独立支撑一次测量。设测量窗口内储能电压由 V_H 降到 V_L ，线性稳压交付负载的是电荷，故容量下限为

$$V_0 C_s (V_H - V_L) \geq E_{\text{measure}}, \quad (4)$$

其中 $V_0 = 3.3 \text{ V}$ ， E_{measure} 为一次约 260 ms 窗口的能耗， V_L 不得低于 LDO 压差加 3.3 V。该式给出的是容量下限，实际取值以窗口内 V_H 、 V_L 实测波形校核。

对增益为 G 的同相偏置放大电路，输出为

$$V_o = GV_i + (1 - G)V_{DAC}. \quad (5)$$

若输入直流中心为 $V_{i,dc}$ ，要求输出中心落在 ADC 中点 V_c ，则 $G > 1$ 时

$$V_{DAC} = \frac{GV_{i,dc} - V_c}{G - 1}. \quad (6)$$

固件以 12 位 ADC 的 2047 码为目标中心，目标峰峰值利用率取满量程的 65%，换档回差的下、上限分别为 25% 和 75%。

TIMG6 以 4 kHz 触发 ADC1，DMA 搬运采样。对 N 点序列 $x[n]$ 施加 Hann 权 $w[n]$ ，加权均值为

$$\bar{x}_w = \frac{\sum_{n=0}^{N-1} w[n]x[n]}{\sum_{n=0}^{N-1} w[n]}, \quad (7)$$

去除直流后的有效值码宽为

$$X_{rms} = \sqrt{\frac{\sum_{n=0}^{N-1} w[n](x[n] - \bar{x}_w)^2}{\sum_{n=0}^{N-1} w[n]}}. \quad (8)$$

加窗是因为 200 ms 记录未必覆盖整数个工频周期，Hann 权把截断处的权重压到零，使非整周期截断不再直接进入结果。各量程分别保存标定点 (x_k, I_k) ，当 $x_k \leq X_{rms} \leq x_{k+1}$ 时按

$$I = I_k + \frac{X_{rms} - x_k}{x_{k+1} - x_k}(I_{k+1} - I_k) \quad (9)$$

分段线性插值。采样点取在整流之后，波形是全波整流后的脉动量，其有效值与原边电流的比例由标定表确定，不依赖对波形因数的解析假设，互感器非线性、采样电阻和链路增益的误差一并被表吸收。

2. 硬件电路设计

系统由电流互感器、SS16 全桥、齐纳箝位与储能、LDO、PMOS/NMOS 分时开关、片内模拟前端、MSPM0G3507、OLED、受控供电的 HC-42 和 Android 读表器组成，框图如图 1 所示。电流表 A、B 结构相同，供能与测量链路互相独立，拆除任一表不影响另一表。完整原理图见附录图 2。

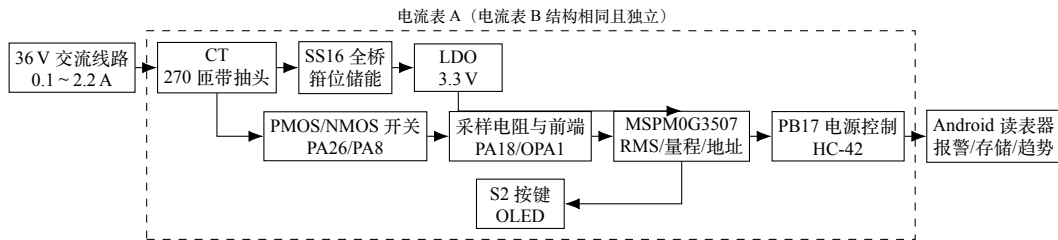


图 1 系统框图

取能支路以 170 匝有效绕组经四只 SS16 组成的全桥整流，正、负半周均向储能电容充电。整流输出记为节点 A，与储能电容之间另串一只肖特基 D_5 ：采集支路接入时把 A 拉低，若无 D_5 ，冷启动积攒的能量会经采集支路反向放掉；有了 D_5 能量只能单向流入电容，测量期间由电容独立维持系统。储能电容并接串了限流电阻的按钮，即题目要求的残余电量放电回路。

采集支路自节点 A 取出，由 PMOS 高边和 NMOS 低边双端把关，采样电阻 R_s 串在两者之间，PMOS 漏极与 R_s 的交点送 PA18。两管同时导通时支路成回路， R_s 上的压降随次级电流变化；任一只关断支路即断开，窗口之外不消耗取能功率。PA26 经 $10\text{k}\Omega$ 上拉至 A，PA8 经 $10\text{k}\Omega$ 下拉至地：上电到 MCU 接管 GPIO 之前引脚是输入而非驱动输出，这两只电阻使支路在该窗口内保持断开，而这段窗口恰好是储能电压最弱、最经不起额外负载的时候。分时控制状态见附录表 3。

模拟前端全部在片内：OPA1 作同相可编程增益放大，DAC12 给出偏置，ADC 和 DAC 的基准取片内 2.5 V 基准而不是供电电压，使供电纹波和 LDO 输出误差不进入读数。引脚分配见附录表 4。

3. 软件设计

程序上电后先把 PA26/PA8 置为取电空闲态，PB17 拉低使蓝牙断电，OLED 不发送初始化命令，随后不等请求先完成一次测量——读表器一连上就有数可读。这次测量结束后才把 PB17 置高、等待 350 ms 并初始化 UART2：模块的启动电流是本固件中最大的单笔负载，不应落在供电刚建立的时刻。

此后主循环等待三件事之一：S2 下降沿、蓝牙收到 MEAS、2 s 心跳定时到期。前两者完全等价，走同一条测量路径；心跳只发一帧保活并继续等待。测量期间到达的请求两者都丢弃而不排队，测量结束时清空串口积压字节，使下一次等待从静默开始。

一次测量的时序是：切换为传感态并等待 20 ms 建立，采 160 点/40 ms 探测帧，按峰峰值和直流中心选定增益与 DAC 偏置（最多 1 ms 建立），再采 800 点/200 ms 正式帧；然后按 TIM6/ADC1、OPA1/DAC12、基准、外部 MOS 开关的顺序关断，最后才计算有效值并查表。关断顺序不可颠倒：先断外部开关会让最后一段采样落在正在塌陷的输入上。程序流程见附录图 4。

读数带质量标志：基准未就绪、输入越出转换窗口、采集帧未完成时，该次读数不可信，固件只发调试帧并给出原因，不发读数帧，避免把不可信的数

值以正常状态送出。

4. 通信协议设计

链路为 HC-42 透传串口 (FFE0 服务、FFE1 通知特征)，速率 115200 bit/s。透传链路不提供分包与长度字段，故采用面向行的 ASCII 协议，以换行作自同步定界符：接收端在任意位置接入或丢失字节后，都能在下一个换行重新对齐。题目 2(2) 规定两种模式下均不得操作电流表，因此读数由读表器请求产生，电流表不主动上报测量值。三种帧如下。

(1) 读表器 → 电流表，请求帧：

MEAS 或 MEAS,ADDR=dd

前者为广播，后者仅编码开关为 dd 的电流表应答。命令带地址而非仅用广播，是因为一台电流表在不同时刻由编码开关赋予不同地址，仅凭连接无法确定对端身份；带地址后请求与应答一一对应，不会把某地址的应答记到另一地址名下。

(2) 电流表 → 读表器，读数帧与调试帧：

METER_TEST,ADDR=dd,CURRENT_MA=iiii,STATUS=ssss

METER_CAL,ADDR=dd,RMS=257.83,GAIN=1,FLAG=OK,MEAN=501,PP=856

CURRENT_MA 为有效值，单位 mA；STATUS 取 NORMAL、LOW、HIGH，按 0.2 A 与 2.0 A 判定，没有 OFFLINE——离线是读表器的判断，电流表无法报告自己不在。调试帧与读数帧同次产生，RMS 和 GAIN 用于建立标定表，MEAN、PP 是探测帧测得的输入直流与峰峰值，FLAG 是该次读数的质量标志。

(3) 电流表 → 读表器，保活帧：

IMHERE,ADDR=dd

空闲时每 2 s 一次，不含读数，也不唤醒任何模拟电路。作用有二：编码开关改变时旧地址的心跳停止、新地址出现，读表器据此更新在场地址；负载断开后电流表随之失电，心跳消失即为题目 2(3) 的离线判据，比等待一次请求超时更快。请求超时只说明本次未读到，不据此判定离线。

5. 其他重要模块设计

OLED 上电不初始化，一次测量结束后显示 1 s 即熄屏，对比度设为最低档，空闲期不刷屏。蓝牙模块不是仅延后 UART，而是由 PB17 控制外部 3.3 V 负载开关，低电平关断、高电平导通；断电期间 UART 引脚保持未初始化，避免经模块 RXD 保护二极管反向灌电。地址由 PB0、PB6 ~ PB8 四位编码开关给出，低

有效，范围 00~15，每次心跳都重读，使运行中拨动开关立即生效。

Android 读表器扫描 8 s、间歇 12 s，连接名称符合 HC-42、METER、AMMETER 等前缀的设备。基本模式为一对一手动读取；自动模式一键启动，每 120 s 对在场地址依次请求，同一连接上只允许一个未完成请求。读数写入 SQLite，容量 1000 条，历史与趋势均超过要求的 30 条；离线与请求超时都不写入记录——记录用于保存读数，缺席不是读数。低于 0.2 A 与高于 2.0 A 的每一次读数都报警，不以状态跳变为条件：读数稀疏，同一状态的相邻两次读数是两个独立事件。

三、 测试方案及测试结果分析

1. 测试仪器与测试条件

测试在室温下进行，由 220 V/36 V、60 VA 单相隔离变压器供电，回路串入标定电流表和自制负载，负载视在功率不超过 50 VA、电流 0~2.2 A 连续可调。仪器见表 1。

表 1 测试仪器列表

序号	仪器类别	仪器型号	数量	用途
1	交流电源	熙硕 STG-500W	1	提供可调交流测试电源
2	五位半万用表	SIGLENT SDM3055X-E	1	标准电流与电压比对
3	数字示波器	SIGLENT SDS2102X PLUS	1	储能、采样与启动时序
4	Android 手机	通用	1	无线读取、报警与存储测试
5	秒表	通用	1	记录启动与通信时间

2. 测试方法

(1) 精度测试：保持线路电压 36 V，依次设置 0.10、0.20、0.50、1.00、1.50、2.00 A，每点稳定后同时记录标准表与被测表读数，每点重复不少于 5 次，A、B 两表分别测试；再拆除 B 表重复测 A，比较误差变化。

(2) 标定与复现性测试：以脚本同步采集 BLE 帧和 SDM3055X-E 读数，负载从低到高再从高到低连续扫描一遍，逐帧记录标准电流、RMS 码宽、量程和质量标志，用于建表并统计帧间散布与来回两趟的系统差。

(3) 取能与启动测试：储能电容经放电按钮充分放电后，从小到大增加负载电流，记录能够稳定完成一次测量并发帧的最低电流，即启动电流；在该电流下用示波器捕获储能电压建立、首次有效值生成和首帧到达的时刻，得到启动

时间。

(4) 分时时序测试：同时观察 PA26、PA8、储能电压和 3.3 V，确认空闲态为 PA26=1、PA8=0，测量态为 PA26=0、PA8=1，并记录 260 ms 传感窗口内的储能电压跌落。

(5) 显示与无线测试：确认上电阶段 OLED 无驱动，单击 S2 后显示 1 s 自动熄屏；手机与电流表相距 5 m 检查人工读取和 120 s 自动模式；设置低限、正常、超限并断开一只表检查报警；写入 30 条以上记录后重启应用，检查历史与趋势。

3. 测试结果

供电绕组：85 匝单段绕组感应电压和调节余量不足；改为 270 匝总绕组、每 50 匝预留抽头后可组合不同有效匝数，经对比最终以 170 匝有效绕组供电。

标定与复现性：2026 年 7 月 31 日的连续扫描共 196 帧，其中 192 帧质量标志为 OK，标准电流覆盖 0.5503 ~ 2.3218 A。归并为 38 个标定点写入 X1 直采档的标定表，覆盖 0.5527 ~ 2.2633 A。表内拟合残差中位数 0.238%、90 分位 0.714%；同一电流点的帧间散布 1σ 为 0.45%；下降趟在同一电流上比上升趟低 0.59%，把两趟合并入表后，上升点残差 +0.17%、下降点残差 -0.24%。剩余 4 帧标志为 OVER：2.33 A 时探测帧测得波形峰值已达 4107 码，越过 4095 上限。

精度记录见表 2，按标准表比对结果逐格填写，不填入推测数据。启动电流、启动时间、5 m 通信距离和外形尺寸同样以实测填写。

表 2 电流测量精度记录表

标准值/A	0.10	0.20	0.50	1.00	1.50	2.00
A 表示值/A	待实测	待实测	待实测	待实测	待实测	待实测
A 表误差/%	待实测	待实测	待实测	待实测	待实测	待实测
B 表示值/A	待实测	待实测	待实测	待实测	待实测	待实测
B 表误差/%	待实测	待实测	待实测	待实测	待实测	待实测
启动电流/A	待实测			启动时间/s	待实测	

4. 结果分析

误差主要来自四处。其一是帧间散布 0.45%：一次读数只用一帧 200 ms 记录，工频与采样窗口不同步、负载本身的波动都落在里面；Hann 加权已经压掉了非整周期截断的边界项，进一步降低要靠多帧取中位数，但那是成倍的采样能耗，与启动电流指标冲突，故未采用。其二是上下行 0.59% 的系统差，其中一部分是比对时序造成的：标准表读数比电流表的采样窗口滞后约 300 ms，扫描

时上升趟与下降趟的偏移方向相反；另一部分来自磁芯磁滞。把两趟合并入同一张表使残差从单向的 0.59% 降到双向的 $+0.17\%/ - 0.24\%$ ，是当前处理方式。

其三是标定覆盖：0.55 A 以下尚无标定点，而该段信号幅度按比例下降，帧间散布的相对值随之放大，是全量程 0.5% 的主要风险；改进措施是在 0.1 ~ 0.55 A 补点，并把采样点改到隔直之后、用固定偏置送入 OPA，使直流中心与信号幅度解耦。当前前端从整流节点取样，直流与交流同比例变化，PGA 会同时放大两者，因此实测中量程始终停在 X1 档——输入自身的摆幅在每个电流下都占直流留出余量的 76% ~ 89%，没有哪一档增益放得进去。其四是上端余量：2.33 A 时峰值已触及 4095 上限，2 A 以上的越限区只有约 15% 宽，改进措施是减小采样电阻或抬高偏置留出的上行余量。

四、 总结

本系统以电流互感器取能，全桥整流、齐纳箝位、电容储能后由低静态电流 LDO 稳压供电，取电与传感用一对 MOS 开关分时复用同一绕组，并以隔离肖特基保证测量窗口内由电容独立供电。测量用片内 OPA、DAC 和 ADC，4 kHz 采样 800 点，Hann 加权真有效值配合分段线性标定表换算，读数经 HC-42 以请求-应答方式送给 Android 读表器。方案可行：0.55 ~ 2.32 A 已建标定表，表内残差中位数 0.238%，分时供电与无线上报均已跑通。优点是空闲期无传感负载、模拟模块全部掉电，有利于启动电流指标，标定表把互感器非线性一并吸收；不足是低端尚未标定、量程档位在当前前端上无法启用，需按上节措施调整采样点位置后重新标定。

附录

1. 硬件电路原理图

图 2 中同名网络（A、3.3 V、GND 以及各引脚名）相连。

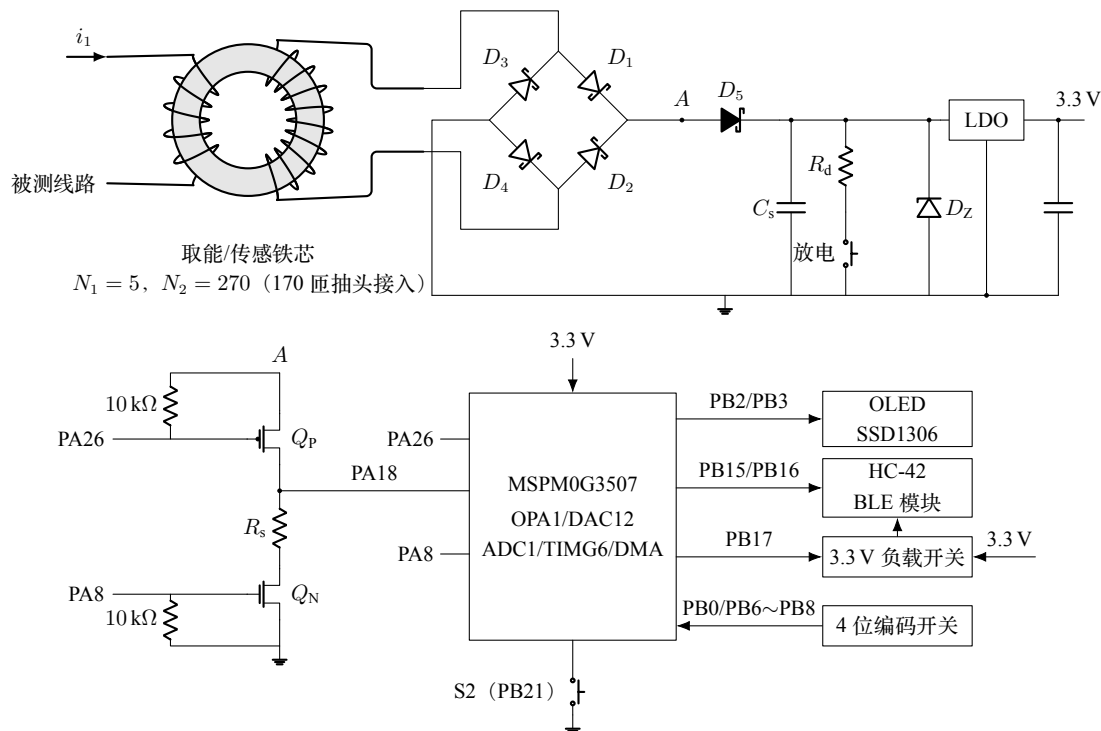


图 2 电流表硬件原理图

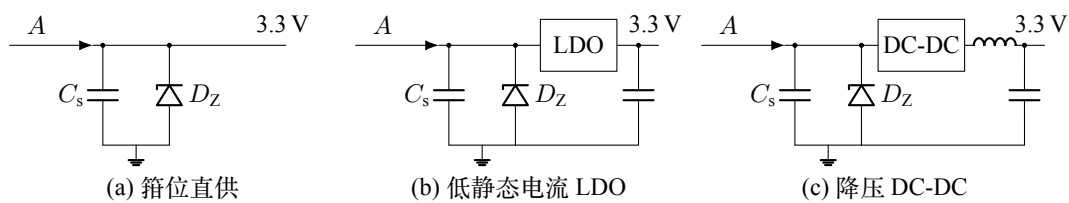


图 3 三种稳压方案

表 3 取电与传感分时控制状态

工作状态	PA26	PA8	MOS 状态	能量与信号路径
空闲/取电	高	低	均关断	传感前端隔离，互感器输出全部用于整流储能
测量/传感	低	高	均导通	传感前端接入，储能电容独立维持系统供电

2. 程序流程图

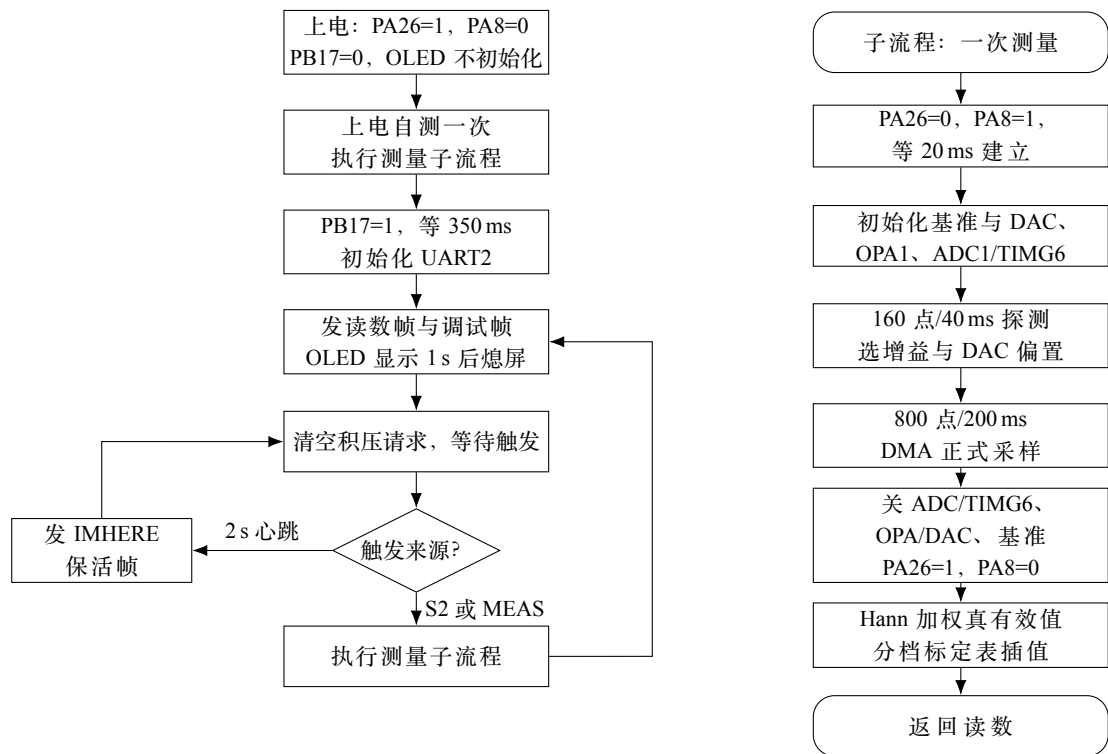


图 4 程序流程图

3. 控制器接口

表 4 MSPM0G3507 主要接口分配

引脚	功能	说明
PA26	PMOS 高边控制	低电平导通；10 k Ω 上拉，上电默认关断
PA8	NMOS 低边控制	高电平导通；10 k Ω 下拉，上电默认关断
PA18	ADC1 输入	采样电阻上的电流采样通道
PA16	OPA1 输出	可编程增益后的模拟通道
PB2、PB3	软件 I ² C	OLED 的 SCL、SDA
PB21	S2 按键	下降沿触发一次测量，与蓝牙 MEAS 等价
PB17	蓝牙电源使能	高电平导通外部负载开关，低电平关闭
PB15	UART2 TX	HC-42 串口发送，115200 bit/s
PB16	UART2 RX	接收读表器的 MEAS 请求
PB0、PB6 ~ PB8	地址输入	4 位低有效地址，范围 00 ~ 15

4. 主要程序代码

电流表固件用 Rust 编写，编译目标为 thumbv6m-none-eabi，异步执行器为 embassy。以下按一次测量经过的顺序列出四个源文件：main.rs 决定何时测量，meter.rs 执行一次测量的完整时序，dsp.rs 把采样序列算成有效值码宽，link.rs 管地址、帧格式和收发。其余模块不在此列出：sampler.rs 配置 TIMG6/ADC1/DMA，range.rs 配置 OPA1 增益与 DAC 偏置并实现换档判据，cal.rs 存分档标定表，vref.rs 开关内部基准，supply.rs 用 ADC0 的内部通道读供电电压，oled.rs 与 ui.rs 驱动显示。行号即源文件行号。

代码 1 src/main.rs: 触发与调度

```
1  #![no_std]
2  #![no_main]
3
4  /// "Passive" AC ammeter, circuit A: one reading per request.
5  ///
6  /// A request is either a press of S2 or a `MEAS` command off the radio; the two
7  /// are the same trigger and take the same path. One request runs one
8  /// `meter::measure`, which switches the external circuit in, takes the reading
9  /// and puts everything back to sleep. The result goes out over the radio and
10 /// onto the panel, which stays lit for `DISPLAY_ON_MS` and then goes dark.
11 ///
12 /// Power-on waits for the supply rail before any of that. Nothing analog is
13 /// enabled and the radio is not powered until ADC0's internal monitor puts VDD
14 /// at `BOOT_MIN_MV`, so a meter coming up on a harvested rail starts once, at a
15 /// voltage every block downstream is rated for, rather than half-starting on the
16 /// way there. Only the LED runs during the wait.
17 ///
18 /// Past that, power-on takes one reading unasked, before the radio is up, and
19 /// reports it the moment the port opens. So the meter always has a number to
20 /// give, and a reader that connects before anyone has pressed anything is not
21 /// told the meter is silent when it is merely idle.
22 ///
23 /// Requests that land inside a cycle are dropped, not queued, from either
24 /// source: `wait_for_falling_edge` clears pending edges before it arms, and
25 /// `Link::discard_pending` throws away buffered command bytes at the same
26 /// point. A reading answers the request that started it.
27 ///
28 /// Between cycles nothing runs -- the executor idles on the two triggers, with
29 /// every analog block powered down and the radio up.
30 ///
31 /// This file is the trigger and nothing else. The substance lives in the
32 /// modules below.
33 /// `meter` -- one reading: AFE control, bring-up, probe, frame, power-down
34 /// `link` -- HC-42 on UART2: the address, the frames out, the command in
35 /// `sampler` -- TIMG6/ADC1/DMA and the sample buffers
36 /// `range` -- OPA1 PGA, DAC bias pivot, and the autoranger
37 /// `dsp` -- windowing, true RMS
38 /// `cal` -- per-range LSB -> amps tables
39 /// `supply` -- the rail itself, off ADC0's internal monitor
40 /// `ui` -- the OLED readout
41 /// `led` -- the power-on indicator
42
43 use embassy_executor::Spawner;
44 use embassy_futures::select::{Either3, select3};
45 use embassy_mspm0::dma::{self, Channel};
46 use embassy_mspm0::gpio::{Input, Level, Output, Pull};
47 use embassy_mspm0::uart::BufferedInterruptHandler;
48 use embassy_mspm0::{bind_interrupts, peripherals};
49 use embassy_time::Timer;
```

```

50 use panic_halt as _;
51
52 mod cal;
53 mod dac;
54 mod dsp;
55 mod led;
56 mod link;
57 mod meter;
58 mod oled;
59 mod range;
60 mod sampler;
61 mod supply;
62 mod ui;
63 mod vref;
64
65 use link::{Address, HEARTBEAT_MS, Link};
66 use meter::Meter;
67 use ui::{DISPLAY_ON_MS, Panel};
68
69 bind_interrupts!(struct Irqs {
70     DMA => dma::InterruptHandler<peripherals::DMA_CH0>;
71     UART2 => BufferedInterruptHandler<peripherals::UART2>;
72 });
73
74 /// Radio buffers. TX only has to hold the two frames of one reading; RX only
75 /// has to hold one command line, and anything past that is a reader talking
76 /// over itself. Both are `static` because `BufferedUart` keeps them for as long
77 /// as the port is open, which here is forever.
78 static mut TX_BUF: [u8; 192] = [0; 192];
79
80 static mut RX_BUF: [u8; 64] = [0; 64];
81
82 /// The rail this meter starts up on, in millivolts. Nothing analog and nothing
83 /// on the radio runs until `supply` measures at least this.
84 ///
85 /// It is a threshold on the *measured* supply, and the measurement carries the
86 /// divider's 1.5% and the reference's 1.4% (datasheet 7.12.1, 7.15.1), so a rail
87 /// regulated to a hair under 3.3 V can read a hair under 3.3 V. With no deadline
88 /// on the wait, that is a meter that never starts: this constant is the one to
89 /// lower.
90 const BOOT_MIN_MV: u16 = 3_300;
91
92 /// Idle until something asks for a reading, announcing this meter's address
93 /// every `HEARTBEAT_MS` in the meantime.
94 ///
95 /// The beat is not a trigger -- it goes out and the wait resumes. Only the
96 /// button and a `MEAS` command return from here, and which one did is not worth
97 /// distinguishing: they are the same request.
98 async fn wait_for_request(trigger: &mut Input<'static>, link: &mut Link, address:
99     ↪ &Address) {
100     loop {
101         let addr = address.read();
102         match select3(
103             trigger.wait_for_falling_edge(),
104             link.wait_command(addr),
105             Timer::after_millis(HEARTBEAT_MS),
106         ) {
107             .await
108             {
109                 Either3::First(()) | Either3::Second(_) => return,
110                 // Re-read the switch on every beat rather than caching it: moving
111                 // the switch is the one operation 2(2) allows during a run, so the
112                 // address has to be able to change between two beats.
113                 Either3::Third(()) => link.send_alive(address.read()),
114             }
115         }
116     }
117 }
118 #[embassy_executor::main]

```

```

118 async fn main(spawner: Spawner) -> ! {
119     let p = embassy_mspm0::init(Default::default());
120
121     // First thing after the clocks, so the indicator is up before anything
122     // that can fail or block: from here on, a dark LED means the firmware
123     // never got this far.
124     //
125     // The task pool is one deep and this is its only spawn, so the token
126     // cannot come back `Err`. Handled rather than unwrapped anyway: an
127     // indicator that failed to start is a reason to run without it, not a
128     // reason to halt a meter that is otherwise fine.
129     if let Ok(token) = led::blink(Output::new(p.PA0, Level::Low)) {
130         spawner.spawn(token);
131     }
132
133     let mut meter = Meter::new(p.PA8, p.PA26);
134     let mut dma = Channel::new(p.DMA_CH0, Irqs);
135     let mut panel = Panel::new();
136     let address = Address::new(p.PB0, p.PB6, p.PB7, p.PB8);
137
138     // LaunchPad S2 is the independent user button on PB21. It pulls the pin
139     // low when pressed; unlike S1/PA18, it does not disturb the ADC input.
140     let mut trigger = Input::new(p.PB21, Pull::Up);
141
142     // Nothing above this line has drawn any current worth the name: the pins are
143     // configured, the LED is blinking, and no analog block has been enabled.
144     // Nothing below it runs until the rail is up.
145     //
146     // This is the whole of the supply gate, and it is here rather than in front
147     // of the display because everything downstream of it wants a rail that has
148     // arrived, not just the SSD1306: the reference the reading is scaled
149     // against needs VDD >= 2.7 V, the radio module wants its 3.3 V, and the
150     // panel's charge pump wants the same. One wait covers all three.
151     //
152     // Being first is also what makes the 1.4 V reference `supply` reads against
153     // free. VREF is untouched at this point, so CVREF starts at 0 V, and the
154     // only crossing the program ever makes afterwards is up to 2.5 V -- never
155     // the 2.5 V -> 1.4 V direction that TRM 23.2.1 needs a discharge for.
156     //
157     // There is no deadline. A rail that never reaches BOOT_MIN_MV parks the
158     // meter here, blinking, which is the honest state to be in: no reading it
159     // could take would be in spec and nothing it drove would be either.
160     supply::wait_above(BOOT_MIN_MV).await;
161
162     // One reading on power-up, before anything can ask for it, and before the
163     // radio exists. Measuring first is what the board notes ask for on a
164     // harvested rail (6.1): the module's start-up current is the largest single
165     // draw in this firmware, and it should not land while the supply has just
166     // come up. Nothing is lost by waiting -- the reading is already in hand
167     // when the port opens, so the first frame goes out as soon as there is
168     // anything to send it over.
169     let mut in_hand = Some(meter.measure(&mut dma).await);
170
171     // SAFETY: this is the only place either buffer is named, and the `Link`
172     // built from them lives for the rest of the program.
173     let mut link = Link::power_up(
174         p.PB17,
175         p.UART2,
176         p.PB15,
177         p.PB16,
178         Irqs,
179         unsafe { &mut *core::ptr::addr_of_mut!(TX_BUF) },
180         unsafe { &mut *core::ptr::addr_of_mut!(RX_BUF) },
181     )
182     .await;
183
184     loop {
185         // The power-on reading is already taken; every later one has to be
186         // asked for. Both then travel the identical path below, so the first

```

```

187 // reading is reported exactly like the ones that follow.
188 let reading = match in_hand.take() {
189     Some(reading) => reading,
190     None => {
191         // Idle here on both triggers at once. Whichever arrives first
192         // wins and the other future is dropped -- for the button that
193         // discards a pending edge, and for the radio it leaves any
194         // buffered bytes where they are, which the discard below then
195         // clears. Which one won does not matter: the two are the same
196         // request.
197         match &mut link {
198             Some(link) => wait_for_request(&mut trigger, link, &address).
↪ await,
199             // A radio that failed to open leaves the meter working off
200             // its button and its panel, which is strictly better than
201             // refusing to measure at all.
202             None => trigger.wait_for_falling_edge().await,
203         }
204         meter.measure(&mut dma).await
205     }
206 };
207
208 // Report before the panel: the reader is waiting on an answer, and the
209 // second the display spends lighting up is a second the radio could
210 // have spent delivering it.
211 if let Some(link) = &mut link {
212     link.send(address.read(), &reading);
213 }
214 panel.show(&reading);
215
216 // Anything that arrived while the meter was busy is dropped here, so
217 // the next `wait_command` starts from silence -- the same rule the
218 // button already follows.
219 if let Some(link) = &mut link {
220     link.discard_pending().await;
221 }
222
223 Timer::after_millis(DISPLAY_ON_MS).await;
224 panel.blank();
225 }
226 }

```

代码 2 src/meter.rs: 一次测量的完整时序

```
1  /// One reading, start to finish: switch the external circuit in, bring the
2  /// analog blocks up, probe, range, take the frame, put everything back to
3  /// sleep, and turn the samples into amps.
4  ///
5  /// Everything true of a reading regardless of what asked for it lives here, so
6  /// the binaries differ only in when they measure and what they do with the
7  /// answer -- main.rs on a button press onto the panel, bin/stream.rs on a
8  /// timer out the radio. Neither can get the sequence subtly different from the
9  /// other, which matters because the order in measure is not arbitrary: the
10 /// calibration tables are only valid for readings taken exactly this way.
11
12 use embassy_msp0::dma::Channel;
13 use embassy_msp0::gpio::{Level, Output};
14 use embassy_msp0::{Peri, peripherals};
15 use embassy_time::Timer;
16
17 use crate::cal::lsb_to_amps;
18 use crate::dsp::{coarse_pivot, rms_lsb};
19 use crate::range::{
20     Gain, OUT_CENTER, Range, apply_range, dac_code_for, next_range, over_range,
21     ↪ power_down_opal,
22     probe_stats, setup_opal_pga,
23 };
24 use crate::sampler::{
25     ADC_CH_RAW_IN, BUF, PINCM_PA16, PINCM_PA18, PROBE_BUF, capture,
26     ↪ init_adcl_event, init_timer,
27     set_adc_channel, set_hiz,
28 };
29
30 /// Settling allowed after the external circuit is switched into its measuring
31 /// state, before the probe frame starts.
32 ///
33 /// What survives into the measurement frame is a decaying offset, and
34 /// rms_lsb subtracts the frame's own weighted mean -- that removes a constant
35 /// but not a decay, so the tail of the turn-on transient lands in the reading.
36 /// Set from the front end's measured turn-on time constant, long enough that
37 /// the residual at the start of the measurement frame is below a LSB.
38 const AFE_SETTLE_MS: u64 = 20;
39
40 /// How much a reading is worth. Ordered by which fault to report when more
41 /// than one is true, worst first -- that ordering is the reason this is one
42 /// enum rather than three flags each consumer re-prioritises for itself.
43 #[derive(Clone, Copy, PartialEq, Eq)]
44 pub enum Quality {
45     /// The 2.5 V reference never reported ready, so every code in the frame is
46     /// against an unsettled reference. Worst of the four because it is the only
47     /// one where the samples look entirely ordinary: they are a plausible
48     /// waveform, scaled by an unknown amount.
49     RefBad,
50     /// The probe found the input pinned at an ADC end, so the signal is
51     /// leaving 0..VDDA and no internal gain or bias choice can recover it --
52     /// that needs external conditioning.
53     InputBad,
54     /// The arithmetic says the signal will not fit even on Direct, the least
55     /// demanding range there is, so the frame is clipping. Every range is
56     /// bounded by the supply and nothing tighter (see range::OUT_LO --
57     /// OPAl's rails and ADC1's full scale are the same two voltages), which
58     /// makes this exactly the same physical condition as InputBad, predicted
59     /// from the probe's mean/pp rather than seen as pinned samples.
60     OverRange,
61     /// One of this cycle's DMA transfers never drained, so its buffer is part
62     /// new samples and part stale ones.
63     Incomplete,
64     Good,
65 }
66
67 /// One measurement. rms and range together are one row of one CAL table,
68 /// which is why they travel with the answer rather than being consumed by it:
```

```

67 /// a reading that cannot say which table produced it cannot be calibrated
68 /// against, and one taken while `quality` is not `Good` must not be entered
69 /// into a table at all -- freezing that fiction into the instrument is worse
70 /// than having no row.
71 pub struct Reading {
72     pub amps: f32,
73     pub rms: f32,
74     pub range: Range,
75     /// Probe mean and peak-to-peak, in input-referred ADC LSB. `None` when the
76     /// probe frame never drained, which is also when the range and the pivot
77     /// are the previous cycle's rather than this one's.
78     #[allow(dead_code)] // Read by bin/stream.rs, not by the panel.
79     pub probe: Option<(i32, u32)>,
80     ref_bad: bool,
81     input_bad: bool,
82     range_over: bool,
83     incomplete: bool,
84 }
85
86 impl Reading {
87     pub fn quality(&self) -> Quality {
88         if self.ref_bad {
89             Quality::RefBad
90         } else if self.input_bad {
91             Quality::InputBad
92         } else if self.range_over {
93             Quality::OverRange
94         } else if self.incomplete {
95             Quality::Incomplete
96         } else {
97             Quality::Good
98         }
99     }
100 }
101
102 pub struct Meter {
103     afe_enable: Output<'static>,
104     afe_enable_n: Output<'static>,
105     /// Survive across cycles because the analog blocks do not: everything is
106     /// powered down between readings, so each cycle rebuilds the front end
107     /// from these two numbers rather than from whatever the registers held.
108     ///
109     /// `range` is where the next `next_range` starts from, and one cycle ago is
110     /// a better guess than the ladder's pessimistic end. `mean_in_last` is the
111     /// input DC the pivot is placed against.
112     range: Range,
113     mean_in_last: i32,
114 }
115
116 impl Meter {
117     /// Claims the external circuit's control pair and puts the two analog pins
118     /// in their idle state. Nothing else in this firmware touches PINCM40 or
119     /// PINCM38.
120     ///
121     /// The initial range and pivot are the choice that assumes least -- x2
122     /// cannot clip anything a higher step would have handled, and centre is the
123     /// placement furthest from both rails. Neither survives the first cycle:
124     /// the probe reads the input at unity gain and `next_range`/`dac_code_for`
125     /// derive both from what it actually measured. Convergence takes one cycle
126     /// rather than several, because the probe's numbers are input-referred and
127     /// so do not depend on which range happens to be selected while it runs.
128     pub fn new(
129         pa8: Peri<'static, peripherals::PA8>,
130         pa26: Peri<'static, peripherals::PA26>,
131     ) -> Self {
132         // The external circuit's two control lines, driven as one complementary
133         // pair: measuring is PA8 high with PA26 low, idle is the reverse. They
134         // are never set independently, which is why `set_measuring` takes a
135         // single bool -- a state where both say "measure" or neither does is

```

```

136 // not a state this circuit has, and the pair should not be able to
137 // express one.
138 //
139 // Both pins need a physical resistor holding their idle level: between
140 // power-on and this line they are inputs, not driven outputs, so PA8
141 // needs a pull-down and PA26 a pull-up. That window is exactly when the
142 // harvested supply has the least to spare, and it is also long -- it
143 // covers the whole boot, not just these two statements.
144 let meter = Self {
145     afe_enable: Output::new(pa8, Level::Low),
146     afe_enable_n: Output::new(pa26, Level::High),
147     range: Range::Pga(Gain::X2),
148     mean_in_last: OUT_CENTER,
149 };
150 set_hiz(PINCM_PA18);
151 set_hiz(PINCM_PA16);
152 meter
153 }
154
155 /// Leaving is the mirror of entering, so the circuit passes through the
156 /// same intermediate state in both directions rather than a different one
157 /// each way.
158 fn set_measuring(&mut self, measuring: bool) {
159     if measuring {
160         self.afe_enable.set_high();
161         self.afe_enable_n.set_low();
162     } else {
163         self.afe_enable_n.set_high();
164         self.afe_enable.set_low();
165     }
166 }
167
168 /// One complete cycle. Returns with every analog block powered down and the
169 /// external circuit back in its idle state, whatever happened in between.
170 pub async fn measure(&mut self, dma: &mut Channel<'_>) -> Reading {
171     self.set_measuring(true);
172
173     // The analog blocks come up inside the front end's own settling window,
174     // so bringing them back costs no latency of its own: the DAC's turn-on
175     // (datasheet 7.17.5, 6.9 us max) and OPAL's enable time (7.19.2, 6 us
176     // max at GBW=HIGHGAIN) are microseconds against AFE_SETTLE_MS here.
177     // Each is a full bring-up, not a resume: the end of every cycle clears
178     // PWREN on all four blocks, and that resets every register in them.
179     //
180     // The gain and the pivot are last cycle's, so a cycle whose probe fails
181     // still measures at a setting that was right rather than at a blind
182     // mid-scale one. The probe replaces both a few milliseconds from now.
183     //
184     // The reference goes first and everything downstream is measured
185     // against it: the DAC's bias, the ADC's full scale, and therefore both
186     // the window the autoranger fits into and the codes the RMS is formed
187     // from. Its 200 us settle (datasheet 7.15.2) hides inside AFE_SETTLE_MS
188     // like the rest.
189     let ref_bad = !crate::vref::init(crate::vref::Output::V2_5);
190     let gain = match self.range {
191         Range::Pga(g) => g,
192         Range::Direct => Gain::X2,
193     };
194     setup_opal_pga(gain, dac_code_for(gain, self.mean_in_last));
195     init_adc1_event();
196     init_timer();
197
198     Timer::after_millis(AFE_SETTLE_MS).await;
199
200     // Both reset every cycle: one cycle is one independent reading, so a
201     // mark can only ever describe the frames this cycle actually took.
202     let mut input_bad = false;
203     let mut range_over = false;
204

```

```

205 // --- Probe: the raw input at unity gain, straight off PA18 ---
206 //
207 // The OPA is not involved and does not move, so this costs one MEMCTL
208 // write and 40 ms, with no settle. Both numbers it yields are
209 // input-referred and therefore independent of the gain currently
210 // selected, which is what lets the range converge in a single cycle
211 // rather than hunting toward it.
212 set_adc_channel(ADC_CH_RAW_IN);
213 // SAFETY: `capture` waits the transfer out (or pauses it) before
214 // returning, so nothing else touches PROBE_BUF concurrently.
215 let probed = capture(dma, unsafe { &mut *core::ptr::addr_of_mut!(PROBE_BUF
↪ ) }).await;
216 let mut probe = None;
217 if probed {
218 // SAFETY: as above -- the transfer is finished or paused.
219 let (mean_in, pp_in, railed) = probe_stats(unsafe { &*core::ptr::
↪ addr_of!(PROBE_BUF) });
220 probe = Some((mean_in, pp_in));
221 input_bad = railed;
222 if !railed {
223 self.mean_in_last = mean_in;
224 self.range = next_range(self.range, mean_in, pp_in);
225 range_over = over_range(self.range, mean_in, pp_in);
226 if apply_range(self.range, mean_in) {
227 // Settle the DAC and the re-tapped ladder before the frame
228 // that carries the marks. Both are microsecond-scale; 1 ms
229 // is slack.
230 // Skipped on Direct, which changes nothing analog.
231 Timer::after_millis(1).await;
232 }
233 }
234 }
235 // A failed or railed probe leaves the range exactly as it was.
236 // Re-ranging on numbers known to be wrong is strictly worse than
237 // keeping a setting that was right one cycle ago.
238
239 // --- Measurement frame: whatever the chosen range put the signal on
240 // (OPA1_OUT on PA16 for the PGA ranges, PA18 itself for Direct, in
241 // which case the channel is already the one the probe just used) ---
242 // SAFETY: `capture` waits the transfer out (or pauses it) before
243 // returning, so nothing else touches BUF concurrently.
244 let framed = capture(dma, unsafe { &mut *core::ptr::addr_of_mut!(BUF) }).
↪ await;
245
246 // Everything below this line works out of BUF, so the whole signal
247 // chain comes down here rather than after the caller is done with the
248 // answer: the front end has no reader left, and the reader has nothing
249 // left to read.
250 //
251 // Order is downstream first. The converter and its sample clock stop,
252 // then the amplifier and its bias reference, then the external circuit
253 // -- so nothing is ever driving into a block that has just lost power,
254 // and the 2.5 V reference outlives both of the blocks that use it.
255 // What this leaves on PA18 is the pad alone: the IOMUX has held it
256 // high-Z since boot, and with OPA1 and ADC1 out of the power domain
257 // there is no longer an analog mux on the other side of it either.
258 crate::sampler::power_down();
259 power_down_opal();
260 self.set_measuring(false);
261
262 // BUF holds 800 hardware-timed samples at 4 kHz (TIM+ADC+DMA, zero CPU
263 // involvement per sample).
264 // SAFETY: as above -- the transfer is finished or paused, so BUF is
265 // ours until the next `measure`.
266 let buf = unsafe { &*core::ptr::addr_of!(BUF) };
267 let pivot = coarse_pivot(buf);
268 let rms = rms_lsb(buf, pivot);
269
270 Reading {

```

```
271         amps: lsb_to_amps(rms, self.range),
272         rms,
273         range: self.range,
274         probe,
275         ref_bad,
276         input_bad,
277         range_over,
278         incomplete: !probed || !framed,
279     }
280 }
281 }
```

代码3 src/dsp.rs: 加窗、真有效值与工频估计

```

1  /// Everything that turns one captured frame into numbers: the window and
2  /// quadrature tables, true RMS, the mains-frequency estimate, and the
3  /// frame-to-frame spread indicator. Pure arithmetic over a sample buffer --
4  /// no registers, no knowledge of which range produced the samples.
5
6  use libm::{atan2f, sqrtf};
7
8  use crate::sampler::{ADC_MASK, N, SAMPLE_HZ};
9
10 const TWO_PI: f32 = 2.0 * core::f32::consts::PI;
11 const PI: f32 = core::f32::consts::PI;
12
13 /// Nominal mains frequency. Only the *correlator* in `estimate_hz` assumes
14 /// this value; the RMS path (`rms_lsb`) does not depend on the mains
15 /// frequency at all -- see the Hann note there.
16 const MAINS_NOM_HZ: u32 = 50;
17
18 /// Samples per nominal mains cycle -- 80. Must divide evenly, so the
19 /// quadrature tables can be indexed by `n % SPP_NOM` instead of carrying a
20 /// running phase (no drift, no per-sample sin/cos on a Cortex-M0+ that has
21 /// no FPU).
22 const SPP_NOM: usize = (SAMPLE_HZ / MAINS_NOM_HZ) as usize;
23
24 /// Half-record length for the two-block phase-difference frequency
25 /// estimate.
26 const HALF: usize = N / 2;
27
28 const _: () = assert!(
29     SAMPLE_HZ.is_multiple_of(MAINS_NOM_HZ),
30     "SPP_NOM must be exact"
31 );
32
33 const _: () = assert!(N.is_multiple_of(2));
34
35 // The whole phase-difference trick rests on the *nominal* fundamental
36 // advancing an exact multiple of  $2\pi$  across HALF samples ( $400 * 2\pi/80 =$ 
37 //  $10\pi$ ). If HALF stopped being a whole number of nominal cycles, the wrapped
38 // phase difference would carry a constant bias instead of reading out pure
39 // frequency deviation.
40 const _: () = assert!(HALF.is_multiple_of(SPP_NOM));
41
42 /// Cosine for const evaluation, argument in radians.
43 ///
44 /// Exists because `libm::cosf` is not a `const fn`, and the tables below are
45 /// worth having as compile-time constants: see the note on `HANN`.
46 ///
47 /// Quadrant-reduced to  $[0, \pi/2]$  and then a Taylor series to  $x^{14}$ . At the
48 /// reduced argument's worst case ( $\pi/2$ ) the first dropped term is  $\sim 1.6e-9$ ,
49 /// well under an f32 ULP, and the whole table checks out at  $1.4e-7$  max
50 /// absolute error against a f64 reference -- about 1.2 ULP at magnitude 1.
51 /// A window function needs nothing like that precision (the Hann sidelobe
52 /// argument in `rms_lsb` would survive  $1e-5$ ), so this has margin to spare.
53 const fn cos_const(x: f32) -> f32 {
54     let mut t = x;
55     while t < 0.0 {
56         t += TWO_PI;
57     }
58     while t >= TWO_PI {
59         t -= TWO_PI;
60     }
61     //  $\cos(2\pi - t) = \cos(t)$ , so fold onto  $[0, \pi]$ .
62     if t > PI {
63         t = TWO_PI - t;
64     }
65     //  $\cos(\pi - t) = -\cos(t)$ , so fold onto  $[0, \pi/2]$  and carry the sign.
66     let sign = if t > PI / 2.0 {
67         t = PI - t;
68         -1.0

```

```

69     } else {
70         1.0
71     };
72
73     let x2 = t * t;
74     let mut term = 1.0f32;
75     let mut sum = 1.0f32;
76     let mut k = lusize;
77     while k <= 7 {
78         term = -term * x2 / ((2 * k - 1) * (2 * k)) as f32;
79         sum += term;
80         k += 1;
81     }
82     sign * sum
83 }
84
85 /// Sine for const evaluation. Same reduction shape as `cos_const`, with its
86 /// own series rather than `cos(pi/2 - x)` -- the subtraction would move the
87 /// rounding error into the argument, where it is worth the most for small x.
88 #[allow(dead_code)] // Retained for the optional bench frequency estimator.
89 const fn sin_const(x: f32) -> f32 {
90     let mut t = x;
91     while t < 0.0 {
92         t += TWO_PI;
93     }
94     while t >= TWO_PI {
95         t -= TWO_PI;
96     }
97     // sin(t + pi) = -sin(t), then sin(pi - t) = sin(t).
98     let mut sign = 1.0f32;
99     if t > PI {
100         t -= PI;
101         sign = -1.0;
102     }
103     if t > PI / 2.0 {
104         t = PI - t;
105     }
106
107     let x2 = t * t;
108     let mut term = t;
109     let mut sum = t;
110     let mut k = lusize;
111     while k <= 7 {
112         term = -term * x2 / ((2 * k) * (2 * k + 1)) as f32;
113         sum += term;
114         k += 1;
115     }
116     sign * sum
117 }
118
119 /// Periodic (DFT-even) Hann, `w[n] = 0.5*(1 - cos(2*pi*n/N))`. The periodic
120 /// form is the one that makes `sum(w)` come out to exactly N/2 and puts the
121 /// window's nulls on the analysis bins, which is the whole point here.
122 ///
123 /// Computed at compile time, so it costs flash instead of SRAM and nothing
124 /// at startup. It used to be built by 401 runtime `cosf` calls (~19 ms on a
125 /// soft-float M0+) into a `static mut`; startup time is a scored item, and
126 /// that was the single largest avoidable chunk of it outside the HC-42's own
127 /// 500 ms settle. Being a plain `static` also retires the last `static mut`
128 /// in this module along with the `unsafe` and the `Tables` handle that
129 /// existed only to contain it.
130 ///
131 /// Still mirrored about n = N/2. At const-eval time that is no longer about
132 /// speed: it makes `HANN[n] == HANN[N - n]` hold bit-for-bit, so the window
133 /// is *exactly* symmetric and cannot bias the phase estimate in `half_phase`.
134 static HANN: [f32; N] = {
135     let mut w = [0.0f32; N];
136     let mut n = 0;
137     while n <= N / 2 {

```

```

138         let v = 0.5 * (1.0 - cos_const(TWO_PI * n as f32 / N as f32));
139         w[n] = v;
140         if n > 0 && n < N / 2 {
141             w[N - n] = v;
142         }
143         n += 1;
144     }
145     w
146 };
147
148 /// One nominal-frequency cycle of quadrature, for the `estimate_hz`
149 /// correlator. Same story as `HANN`: compile-time, flash-resident.
150 #[allow(dead_code)] // Retained for the optional bench frequency estimator.
151 static COS_TAB: [f32; SPP_NOM] = {
152     let mut t = [0.0f32; SPP_NOM];
153     let mut n = 0;
154     while n < SPP_NOM {
155         t[n] = cos_const(TWO_PI * n as f32 / SPP_NOM as f32);
156         n += 1;
157     }
158     t
159 };
160
161 #[allow(dead_code)] // Retained for the optional bench frequency estimator.
162 static SIN_TAB: [f32; SPP_NOM] = {
163     let mut t = [0.0f32; SPP_NOM];
164     let mut n = 0;
165     while n < SPP_NOM {
166         t[n] = sin_const(TWO_PI * n as f32 / SPP_NOM as f32);
167         n += 1;
168     }
169     t
170 };
171
172 /// How many recent frames the spread indicator remembers.
173 #[allow(dead_code)] // Retained for bench noise characterization.
174 pub const SPREAD_N: usize = 16;
175
176 /// Peak-to-peak spread of the last SPREAD_N raw frame RMS values, in LSB.
177 ///
178 /// Deliberately *displayed* rather than *filtered*. Cross-frame averaging was
179 /// tried here and removed: any IIR/running mean carries hidden state whose
180 /// lag is invisible on the display, and it misbehaves exactly in the band
181 /// that matters. A step smaller than the reset threshold but larger than the
182 /// 0.5% spec (say a 1.00 -> 1.01 A load nudge) is not detected as a change,
183 /// so the reading creeps toward it over tens of frames while the settle
184 /// indicator claims to be settled. A bench DMM has an aperture (NPLC) and
185 /// emits one reading per aperture; it keeps no memory across readings. If
186 /// this board turns out to need less noise, the correct lever is a longer
187 /// aperture -- accumulate M consecutive captures into one reading, which is
188 /// bounded, has no lag beyond the aperture itself, and stays fully valid one
189 /// aperture after any load change.
190 ///
191 /// What this gives instead: the frame-to-frame spread, on the actual bench,
192 /// with no noise model assumed. That is the sigma measurement that decides
193 /// whether a longer aperture is needed at all -- and it comes for free from
194 /// the display, with no separate procedure.
195 #[allow(dead_code)] // Retained for bench noise characterization.
196 pub struct Spread {
197     hist: [f32; SPREAD_N],
198     idx: usize,
199     len: usize,
200 }
201
202 #[allow(dead_code)] // Retained for bench noise characterization.
203 impl Spread {
204     pub const fn new() -> Self {
205         Self {
206             hist: [0.0; SPREAD_N],

```

```

207         idx: 0,
208         len: 0,
209     }
210 }
211
212 /// Records one frame and returns the current peak-to-peak spread.
213 pub fn push(&mut self, x: f32) -> f32 {
214     self.hist[self.idx] = x;
215     self.idx = (self.idx + 1) % SPREAD_N;
216     if self.len < SPREAD_N {
217         self.len += 1;
218     }
219
220     let mut lo = self.hist[0];
221     let mut hi = self.hist[0];
222     for &v in &self.hist[..self.len] {
223         if v < lo {
224             lo = v;
225         }
226         if v > hi {
227             hi = v;
228         }
229     }
230     hi - lo
231 }
232 }
233
234 // NOTE ON THE NOISE BIAS -- deliberately *not* corrected here.
235 //
236 // Noise adds to the signal in quadrature, so the raw estimate reads
237 //  $\sqrt{\text{signal}^2 + \text{sigma}^2}$ , high by about  $\text{sigma}^2/(2 \cdot \text{rms}^2)$ . It is a
238 // one-sided bias rather than spread, so frame averaging and any smoothing
239 // filter leave it untouched. At 0.1 A (rms ~29.1 LSB) it is worth 0.06% for
240 // sigma = 1 LSB and 0.53% for sigma = 3 LSB; at 2 A it is 400x smaller.
241 //
242 // It must NOT be corrected with a hardcoded sigma^2. The harvesting front
243 // end has least available power at light load, so its ripple is worst
244 // exactly where the spec is tightest -- sigma is a function of load current,
245 // not a constant. Simulating sigma rising 1 LSB (at 2 A) to 4 LSB (at 0.1 A):
246 // a two-point k/b fit alone leaves 0.23% worst-case, while subtracting a
247 // fixed sigma^2 = 9 leaves 0.41%. A constant correction is worse than none.
248 //
249 // What CAL_OFFSET does instead: a two-point fit absorbs most of this bias,
250 // because b lands near  $-\text{sigma}^2/(2 \cdot \text{rms\_lo})$  by construction. Residual is a
251 // bowl, zero at both fit points, worst near 0.2 A -- 0.12% at sigma = 3 LSB,
252 // 0.33% at 5, 1.26% at 10. So the bench measurement to make first is sigma
253 // *at several load currents*: if it stays under ~3 LSB across the range,
254 // CAL_OFFSET alone provably covers it and nothing more is needed. If it does
255 // not, the fix is to estimate the noise per frame at runtime (its power is
256 // separable -- all real load current sits on harmonics of the line
257 // frequency, so inter-harmonic bins such as 75/125/175 Hz see noise only),
258 // never to freeze a number into flash.
259 //
260 // To measure sigma: open the load, leave everything else powered exactly as
261 // in normal operation (the rectifier and supercap charging are the suspected
262 // source, so they must be live), and read the raw `rms` line off the OLED --
263 // that reading *is* sigma in LSB. Repeat under load at several currents by
264 // comparing the reading's spread against the reference ammeter.
265 //
266 /// Coarse DC pivot, integer and exact-to-1-LSB. Deliberately *not* the DC
267 /// estimate: it only has to land within a few LSB of the ~2048-count OPA
268 /// bias so the f32 accumulators downstream never hold a large sum whose
269 /// small difference matters. Its truncation error is solved out exactly by
270 /// the weighted offset in `rms_lsb`, which is why the old integer-division
271 /// mean (up to 1 LSB off, worth a one-sided ~0.06% RMS bias at 0.1 A) is
272 /// no longer a problem.
273 pub fn coarse_pivot(buf: &[u32; N]) -> i32 {
274     (buf.iter().map(|&x| (x & ADC_MASK) as u64).sum::<u64>() / N as u64) as i32
275 }

```

```

276
277 /// Hann-weighted true RMS over one captured frame, in raw ADC-LSB units.
278 ///
279 /// Why weighted: a plain rectangular average of  $x^2$  is only unbiased when
280 /// the window spans a whole number of mains cycles. The error term is the
281 ///  $\star weight \star$  sequence's frequency response evaluated at  $2f$  -- bin 20 of this
282 /// 200 ms record. A rectangular window's sidelobes there are only  $\sim -35$  dB,
283 /// which at 49.66 Hz is a 0.33% RMS error; Hann's are below  $-95$  dB, so the
284 /// term vanishes. That happens without knowing the mains frequency at all,
285 /// which also makes it immune to SAMPLE_HZ itself being wrong (SYSOSC is an
286 /// internal RC,  $\sim \pm 1\%$ , and only the ratio  $f/SAMPLE\_HZ$  ever mattered).
287 /// Harmonic  $h$  lands at bin  $20h$ , i.e. suppressed even harder, so distorted
288 /// load currents are covered by the same mechanism -- and no zero crossing
289 /// is needed, so a dead-zone waveform is fine too.
290 ///
291 /// Cost: effective noise bandwidth  $\times 1.5$ , taking the per-frame noise floor
292 /// from  $\sim 0.035\%$  to  $\sim 0.043\%$  at 0.1 A. Irrelevant against a 0.5% budget.
293 ///
294 /// TESTING TRAP: do not bench this with a signal generator set to exactly
295 /// 50.000 Hz. At exactly SAMPLE_HZ/SPP_NOM the record contains only 80
296 /// distinct signal phases repeated 10 times, so the quantization error is
297 /// perfectly correlated with the signal and does not average down at all --
298 /// simulation puts the worst-phase error at 0.51% at 0.1 A, and no window
299 /// can fix it because it is not leakage. Real mains never sits exactly on
300 /// 50.000 Hz (and SYSOSC guarantees SAMPLE_HZ does not either), so the
301 /// incoherent case is the real one; at 49.8 Hz the same test gives 0.07%.
302 /// A generator at 50.02 Hz reproduces field behaviour, 50.000 does not.
303 pub fn rms_lsb(buf: &[u32; N], pivot: i32) -> f32 {
304     // Pass 1: weighted mean of the pivot-centred residual. The mean has to
305     // be weighted by the  $\star same \star$  window as the sum of squares -- an
306     // unweighted mean leaves fundamental residue that Hann cannot then
307     // suppress, and re-introduces the error this function exists to remove.
308     let mut sum_w = 0.0f32;
309     let mut sum_wd = 0.0f32;
310     for (&x, &w) in buf.iter().zip(HANN.iter()) {
311         sum_w += w;
312         sum_wd += w * ((x & ADC_MASK) as i32 - pivot) as f32;
313     }
314     // True weighted DC, expressed as an offset from the integer pivot.
315     let offset = sum_wd / sum_w;
316
317     // Pass 2. Two passes are mandatory, not stylistic: folding this into
318     // pass 1 as  $\text{sum}(w \star x^2) / \text{sum}(w) - \text{mean}^2$  subtracts two  $\sim 4.2e6$  quantities
319     // to recover  $\sim 848$ , and f32 has no mantissa left for that cancellation.
320     let mut sum_wd2 = 0.0f32;
321     for (&x, &w) in buf.iter().zip(HANN.iter()) {
322         let d = ((x & ADC_MASK) as i32 - pivot) as f32 - offset;
323         sum_wd2 += w * d * d;
324     }
325
326     sqrtf(sum_wd2 / sum_w)
327 }
328
329 /// Phase of the MAINS_NOM_HZ component of one HALF-sample block, measured
330 /// at the block's midpoint, via direct quadrature correlation.
331 ///
332 /// For  $x[n] = A \sin(w \star n + \phi)$  correlated against the nominal  $w0$ :
333 ///  $i\_acc \sim (A/2) \star \text{sum}(w) \star \sin(\phi)$ ,  $q\_acc \sim (A/2) \star \text{sum}(w) \star \cos(\phi)$ 
334 /// hence  $\text{atan2}(i, q)$ . Hann-weighted first: the image term at  $2 \star f0$  sits
335 /// only 10 bins away in a 400-sample block, where a rectangular window
336 /// leaks  $\sim 3\%$  and would swing the phase by up to  $\sim 0.03$  rad -- about 0.05 Hz.
337 /// A 400-point Hann is exactly the 800-point table decimated by 2.
338 ///
339 ///  $\text{start}$  indexes the record, but the correlator is driven by the  $\star local \star$ 
340 /// index, which is what makes the two blocks' phase difference come out as
341 /// the fundamental's advance across HALF samples.
342 #[allow(dead_code)] // Retained for the optional bench frequency estimator.
343 pub fn half_phase(buf: &[u32; N], start: usize, pivot: i32) -> f32 {
344     let mut i_acc = 0.0f32;

```

```

345     let mut q_acc = 0.0f32;
346     for m in 0..HALF {
347         let d = ((buf[start + m] & ADC_MASK) as i32 - pivot) as f32 * HANN[2 * m];
348         let p = m % SPP_NOM;
349         i_acc += d * COS_TAB[p];
350         q_acc += d * SIN_TAB[p];
351     }
352     atan2f(i_acc, q_acc)
353 }
354
355 /// Mains frequency from the phase the fundamental advances between the two
356 /// halves of one record.
357 ///
358 /// Across HALF samples the *nominal* fundamental advances  $400 \times 2\pi / 80 = 10\pi$ 
359 /// -- an exact multiple of  $2\pi$  -- so the wrapped phase difference is purely
360 /// the deviation:
361 ///
362 ///  $d\phi = 2\pi * (f - 50) * HALF / SAMPLE\_HZ = 0.6283 * (f - 50)$ 
363 ///
364 /// Unambiguous over  $\pm 5$  Hz, far wider than the grid ever wanders. This uses
365 /// all 800 samples with SNR-optimal weighting instead of the ~18 samples
366 /// nearest zero, so it neither cares about noise at the zero crossing nor
367 /// can be fooled by a spurious extra crossing.
368 ///
369 /// Reported for the display and the design report only -- `rms_lsb` does
370 /// not consume it, so a bad frequency estimate cannot corrupt the accuracy
371 /// figure that carries the marks.
372 #[allow(dead_code)] // Retained for the design report and bench diagnostics.
373 pub fn estimate_hz(buf: &[u32; N], pivot: i32) -> f32 {
374     let p1 = half_phase(buf, 0, pivot);
375     let p2 = half_phase(buf, HALF, pivot);
376
377     let mut dphi = p2 - p1;
378     while dphi > core::f32::consts::PI {
379         dphi -= TWO_PI;
380     }
381     while dphi <= -core::f32::consts::PI {
382         dphi += TWO_PI;
383     }
384
385     let phi_per_hz = TWO_PI * HALF as f32 / SAMPLE_HZ as f32;
386     MAINS_NOM_HZ as f32 + dphi / phi_per_hz
387 }

```

代码 4 src/link.rs: 地址、帧格式与收发

```
1  /// The wireless link: HC-42 on UART2, the address switch that names this
2  /// meter, the frames that go out and the one command that comes in.
3  ///
4  /// Shared by both binaries so there is exactly one wire format. `main.rs`
5  /// keeps the link up and measures when told to; `bin/stream.rs` keeps it up and
6  /// measures on a timer. Neither can drift from the other's grammar.
7
8  use core::fmt::Write as _;
9
10 use embassy_mspm0::Peri;
11 use embassy_mspm0::gpio::{Input, Level, Output, Pull};
12 use embassy_mspm0::interrupt::typelevel::Binding;
13 use embassy_mspm0::peripherals;
14 use embassy_mspm0::uart::{BufferedInterruptHandler, BufferedUart, Config as
    ↪ UartConfig, Instance};
15 use embassy_time::Timer;
16 use embedded_io_async::{Read, ReadReady};
17
18 use crate::meter::{Quality, Reading};
19
20 /// HC-42 settling after its supply comes up, before UART2 exists. The module
21 /// must not see the idle-high TX line while unpowered -- current would flow
22 /// back through its RXD protection diode -- so this delay is also what
23 /// separates "supply up" from "pin driven".
24 const BT_SETTLE_MS: u64 = 350;
25
26 /// Not the HC-42's factory 9600: at that rate one reading's two frames are 110
27 /// bytes and 115 ms of pure wire time, which lands directly in how long a
28 /// reader waits for an answer. 115200 makes the same frames 9.6 ms and costs
29 /// nothing else -- the module supports up to 230400 (HC42.pdf 5.3.5).
30 ///
31 /// **The module must be set to match**, with `AT+UART=115200`; `bin/btcfg.rs`
32 /// does it and reads the value back. A module left at 9600 is not slow, it is
33 /// silent, which is the failure this constant exists to make visible: change it
34 /// here and the meter goes mute until the module is changed too.
35 pub const BT_BAUD_RATE: u32 = 115_200;
36
37 /// Alarm thresholds from 2(3), in mA. The phone classifies again from its own
38 /// preferences; sending our own classification means a meter read by anything
39 /// else still says what it thinks.
40 const LOW_LIMIT_MA: u32 = 200;
41
42 const HIGH_LIMIT_MA: u32 = 2_000;
43
44 /// `METER_TEST` allows 1-7 digits. A reading that would overflow that is
45 /// nonsense anyway, and a malformed frame is worse than a saturated one: the
46 /// parser drops it whole, so the fault would look like silence.
47 const MAX_FRAME_MA: u32 = 9_999_999;
48
49 /// Longest command line accepted. Anything longer is a reader that has lost
50 /// the plot or line noise that happens to lack newlines; either way the buffer
51 /// resets rather than growing.
52 const CMD_MAX: usize = 32;
53
54 /// How often the meter announces itself while idle. Cheap enough to ignore:
55 /// one 17-byte line is 18 ms of 9600-baud UART and no analog blocks at all,
56 /// against the 1.22 mA the HC-42 already draws holding a full-speed connection
57 /// (HC42.pdf 1.3).
58 ///
59 /// It is what makes the reader's view of "who is out there" track the coded
60 /// switch: one meter stands in for 16 addresses, and when the switch moves, the
61 /// old address simply stops announcing and the new one starts. It also makes a
62 /// disconnected load visible within a few seconds -- the meter loses power with
63 /// it, so the announcements just stop -- which is exactly the offline condition
64 /// of 2(3), and far quicker than waiting for a poll to time out.
65 pub const HEARTBEAT_MS: u64 = 2_000;
66
67 /// What a reader can ask for. One variant today, and the parser is written so
```

```

68 /// that stays cheap to extend.
69 #[derive(Clone, Copy, PartialEq, Eq)]
70 pub enum Command {
71     /// Take one reading now. Equivalent to a press of S2, including what
72     /// happens to a second one that arrives mid-measurement: nothing.
73     Measure,
74 }
75
76 /// The four-position coded switch, ON to ground against internal pull-ups, so
77 /// a closed switch reads low and contributes its weight.
78 ///
79 /// Read per frame rather than latched at boot: the address is meant to be set
80 /// on the bench with the meter running, and a value that only takes effect
81 /// after a power cycle is a value that gets set wrong once and stays wrong.
82 pub struct Address {
83     bit0: Input<'static>,
84     bit1: Input<'static>,
85     bit2: Input<'static>,
86     bit3: Input<'static>,
87 }
88
89 impl Address {
90     pub fn new(
91         pb0: Peri<'static, peripherals::PB0>,
92         pb6: Peri<'static, peripherals::PB6>,
93         pb7: Peri<'static, peripherals::PB7>,
94         pb8: Peri<'static, peripherals::PB8>,
95     ) -> Self {
96         Self {
97             bit0: Input::new(pb0, Pull::Up),
98             bit1: Input::new(pb6, Pull::Up),
99             bit2: Input::new(pb7, Pull::Up),
100             bit3: Input::new(pb8, Pull::Up),
101         }
102     }
103
104     pub fn read(&self) -> u8 {
105         (self.bit0.is_low() as u8)
106         | (self.bit1.is_low() as u8) << 1
107         | (self.bit2.is_low() as u8) << 2
108         | (self.bit3.is_low() as u8) << 3
109     }
110 }
111
112 pub struct Link {
113     uart: BufferedUart<'static>,
114     /// Partial command line, carried across reads because a 9600-baud line
115     /// arrives a few bytes at a time.
116     pending: heapless::String<CMD_MAX>,
117     _power: Output<'static>,
118 }
119
120 impl Link {
121     /// Power the radio and open the port, in that order and with the settle in
122     /// between (board notes 6.1). Stays up from here: a module that has lost
123     /// power cannot be woken over the air, so the only thing that could turn it
124     /// back on is the local button -- which defeats the point of accepting
125     /// commands at all.
126     ///
127     /// PB17 needs an external pull-down for the window before this runs, or the
128     /// radio powers up during flashing and reset.
129     pub async fn power_up<T: Instance>(
130         pb17: Peri<'static, peripherals::PB17>,
131         uart: Peri<'static, T>,
132         tx: Peri<'static, impl embassy_mspm0::uart::TxPin<T>>,
133         rx: Peri<'static, impl embassy_mspm0::uart::RxPin<T>>,
134         irq: impl Binding<T::Interrupt, BufferedInterruptHandler<T>>,
135         tx_buf: &'static mut [u8],
136         rx_buf: &'static mut [u8],

```

```

137     ) -> Option<Self> {
138         let power = Output::new(pb17, Level::High);
139         Timer::after_millis(BT_SETTLE_MS).await;
140
141         let mut config = UartConfig::default();
142         config.baudrate = BT_BAUD_RATE;
143         let uart = BufferedUart::new(uart, tx, rx, irq, tx_buf, rx_buf, config).ok
↳ ()?;
144         Some(Self {
145             uart,
146             pending: heapless::String::new(),
147             _power: power,
148         })
149     }
150
151     /// Wait for a command. Bytes that arrive while nobody is waiting are held
152     /// by the buffered UART, so a command sent a moment early is not lost --
153     /// only one sent during a measurement is, and that is deliberate: see
154     /// `discard_pending`.
155     pub async fn wait_command(&mut self, addr: u8) -> Command {
156         let mut byte = [0u8; 1];
157         loop {
158             if self.uart.read(&mut byte).await.is_err() {
159                 continue;
160             }
161             if let Some(command) = self.feed(byte[0], addr) {
162                 return command;
163             }
164         }
165     }
166
167     /// Throw away everything buffered, partial line included.
168     ///
169     /// Called after a measurement so a command that arrived during one is
170     /// dropped rather than queued -- the same rule the button follows, where
171     /// `wait_for_falling_edge` clears pending edges before it arms. A reading
172     /// answers the request that started it; a request made while the meter was
173     /// busy would otherwise be answered by a reading it did not ask for, and
174     /// the reader has no way to tell the two apart.
175     pub async fn discard_pending(&mut self) {
176         self.pending.clear();
177         let mut sink = [0u8; 32];
178         // `read_ready` is what keeps the await from parking: it only runs when
179         // there is something buffered, so this drains and returns rather than
180         // waiting for a byte that may never come.
181         while self.uart.read_ready().unwrap_or(false) {
182             if self.uart.read(&mut sink).await.is_err() {
183                 break;
184             }
185         }
186     }
187
188     /// One byte into the line assembler. Returns a command on the newline that
189     /// completes one.
190     fn feed(&mut self, byte: u8, addr: u8) -> Option<Command> {
191         if byte == b'\n' || byte == b'\r' {
192             let command = parse_command_for(self.pending.trim(), addr);
193             self.pending.clear();
194             return command;
195         }
196         // A line that outgrows the buffer is dropped rather than truncated:
197         // truncation can turn an unknown command into a known prefix of one.
198         if self.pending.push(byte as char).is_err() {
199             self.pending.clear();
200         }
201         None
202     }
203
204     /// "I am here, and I am meter n." Carries no reading on purpose: this is a

```

```

205     /// presence beat, not a measurement, and nothing analog wakes up for it.
206     pub fn send_alive(&mut self, addr: u8) {
207         let _ = writeln!(self, "IMHERE,ADDR={}", addr);
208     }
209
210     /// One reading, on the wire.
211     ///
212     /// `METER_TEST` is withheld when the reading is one the meter cannot stand
213     /// behind, because that frame has nowhere to say so -- its STATUS field is
214     /// the alarm classification, and sending a fiction as NORMAL is worse than
215     /// sending nothing: a reader that stops hearing from a meter shows it
216     /// offline, which is true, while a confident wrong number is not.
217     ///
218     /// `OverRange` is the exception. It means the signal will not fit even at
219     /// unity gain, which on this front end is a current past the top of scale
220     /// -- the number is understated but the direction is known, and that is
221     /// exactly the case 2(3) wants alarmed. It goes out as HIGH.
222     ///
223     /// `METER_CAL` always goes out, including for the withheld cases. A frame
224     /// the bench cannot use is still evidence, and it carries its own FLAG.
225     pub fn send(&mut self, addr: u8, reading: &Reading) {
226         let quality = reading.quality();
227         let ma = milliamps(reading);
228
229         let test_status = match quality {
230             Quality::Good => Some(status(ma)),
231             Quality::OverRange => Some("HIGH"),
232             Quality::RefBad | Quality::InputBad | Quality::Incomplete => None,
233         };
234         if let Some(test_status) = test_status {
235             let _ = writeln!(
236                 self,
237                 "METER_TEST,ADDR={},CURRENT_MA={},STATUS={}",
238                 addr, ma, test_status
239             );
240         }
241
242         let _ = write!(
243             self,
244             "METER_CAL,ADDR={},RMS={:.2},GAIN={},FLAG={}",
245             addr,
246             reading.rms,
247             reading.range.nominal(),
248             flag(quality)
249         );
250         // Omitted rather than zeroed when the probe frame never drained: a mean
251         // of 0 is a legal reading, and a bench that cannot tell it from "no
252         // probe" would chase the wrong fault.
253         if let Some((mean_in, pp_in)) = reading.probe {
254             let _ = write!(self, "MEAN={},PP={}", mean_in, pp_in);
255         }
256         let _ = writeln!(self);
257     }
258 }
259
260 /// `core::fmt` sink, so frames are written with plain `write!` instead of
261 /// formatting into a `heapless::String` a line at a time.
262 impl Link {
263     /// Push every byte, however many calls that takes.
264     ///
265     /// `BufferedUartTx::blocking_write` is a *short* write: it copies into the
266     /// ring buffer's contiguous region and returns how much fit, which is less
267     /// than asked for whenever the ring wraps mid-frame. Ignoring that count
268     /// truncates frames at a boundary that moves with traffic, and a truncated
269     /// frame does not read as a fault -- the parser simply drops it, so the
270     /// meter looks intermittently silent instead of broken.
271     fn put(&mut self, mut bytes: &[u8]) {
272         while !bytes.is_empty() {
273             match self.uart.blocking_write(bytes) {

```

```

274         Ok(0) | Err(_) => return,
275         Ok(n) => bytes = &bytes[n..],
276     }
277 }
278 }
279 }
280
281 /// `core::fmt` sink, so frames are written with plain `write!` instead of
282 /// formatting into a `heapless::String` a line at a time.
283 impl core::fmt::Write for Link {
284     /// Translates bare LF to CRLF. `MeterFrameParser` strips the CR itself, but
285     /// every serial terminal used to look at this link mid-bench does not.
286     fn write_str(&mut self, s: &str) -> core::fmt::Result {
287         for part in s.split_inclusive('\n') {
288             match part.strip_suffix('\n') {
289                 Some(head) => {
290                     self.put(head.as_bytes());
291                     self.put(b"\r\n");
292                 }
293                 None => self.put(part.as_bytes()),
294             }
295         }
296         Ok(())
297     }
298 }
299
300 /// `MEAS` alone is a broadcast; `MEAS,ADDR=n` is ignored unless `n` is this
301 /// meter's switch setting, so a reader can talk to one of several in range.
302 ///
303 /// Deliberately not `METER_TEST`-shaped: commands travel the opposite
304 /// direction on the same transparent link, and a grammar a meter could mistake
305 /// for its own output is one echo away from a meter triggering itself.
306 pub fn parse_command_for(line: &str, addr: u8) -> Option<Command> {
307     let command = parse_command(line)?;
308     match line.split_once(",ADDR=") {
309         None => Some(command),
310         Some((), want) => (want.trim().parse::<u8>().ok()? == addr).then_some(
311             ↪ command),
312     }
313 }
314
315 fn parse_command(line: &str) -> Option<Command> {
316     let head = line.split(',').next()?;
317     match head {
318         "MEAS" => Some(Command::Measure),
319         _ => None,
320     }
321 }
322
323 fn milliamps(reading: &Reading) -> u32 {
324     // f32 -> u32 saturates at both ends in Rust, so a negative RMS (which the
325     // CAL extrapolation can produce below the table's first point) lands on 0
326     // rather than wrapping to something enormous.
327     ((reading.amps * 1000.0) as u32).min(MAX_FRAME_MA)
328 }
329
330 fn status(milliamps: u32) -> &'static str {
331     if milliamps < LOW_LIMIT_MA {
332         "LOW"
333     } else if milliamps > HIGH_LIMIT_MA {
334         "HIGH"
335     } else {
336         "NORMAL"
337     }
338 }
339
340 fn flag(quality: Quality) -> &'static str {
341     match quality {
342         Quality::RefBad => "REF_BAD",

```

```
342         Quality::InputBad => "BAD_INPUT",
343         Quality::OverRange => "OVER",
344         Quality::Incomplete => "PARTIAL",
345         Quality::Good => "OK",
346     }
347 }
```